SOTER: Guarding Black-box Inference for General Neural Networks at the Edge

Anonymous Author(s)

Abstract

Tremendous AI applications powered by deep learning inference are increasingly deployed on enormous edge devices to provide an interactive user experience. This necessitates securely offloading the proprietary model from the cloud to the edge, and using a co-located accelerator (e.g., GPU) to speed up model inference locally. A prevalent approach is to use publicly available Trusted Execution Environments (TEEs) in CPU to protect model confidentiality from the untrusted software stack. However, despite much effort, securely outsourcing inference computations from a TEE to a faster, yet untrusted GPU remains a fundamental challenge. Existing TEE-based solutions either ban the choice of GPU acceleration or take a handcrafted approach to partition insensitive model layers to run in a GPU. While the latter approach achieves better performance, it requires substantial expertise for model partitioning and is only safe for specific models.

We propose SOTER, the first system that protects confidentiality while achieving low inference latency for general DNN models. Our key observation is that there is often an *associativity* property among many inference operators in a model. Therefore, SOTER automatically profiles and transforms a major fraction of operators into *parameter-morphed* and *confidentiality-preserving* operators to execute in an untrusted GPU, and fully recovers execution results into accurate results within TEE. Based on these steps, SOTER further designs an *oblivious fingerprinting* technique to safely detect integrity breaches of these morphed operators outside TEE. Extensive evaluation of SOTER and two notable systems with nine prevalent models in four most popular categories shows that: SOTER achieves a high confidentiality score in terms of a well-known metric and can effectively detect integrity breaches with high probability; SOTER achieves comparable performance with handcrafted partition-based systems.

1 Introduction

The prosperity of AI [23] and edge computing [15] has pushed more and more well-trained proprietary Deep Neural Network models to be deployed on enormous third-party edge devices to compose mission-critical applications, such as autopilot navigation [8, 13], home monitoring [18] and visual assistance [59]. These applications run *model inference* to conduct real-time data analysis, which typically requires *millisecond level* responses [27, 81] for interactive user experience.

Since the model providers (e.g., Google) are often different from the owners of edge devices (e.g., Samsung or Xiaomi), offloading proprietary models to untrusted edge devices poses a threat to model confidentiality. As obtaining an accurate model requires the model providers (e.g., Google) to pay substantial resources to design an optimal model architecture and train model parameters [55], if directly publishing a model to run inference at the edge, a corrupted client (owner) of edge device can observe and steal the proprietary model architecture and parameters [52, 87].

Hence, there is an urgent demand to achieve low inference latency while preserving model confidentiality with *black-box* accesses at the edge. *Black-box* means that a client can query the model as an oracle but will learn nothing about the sensitive model architecture and parameters.

Although there are some promising trusted GPU research systems [5, 34, 57, 83] that can protect model confidentiality and accelerate inference with the strong GPU computation power, these systems are not publicly available as they require extensive hardware modifications [83] or support only hardwarer simulators [5, 34, 57]. Thus, in this paper, we consider only publicly available (untrusted) GPU deployments.

A recent pragmatic approach is to leverage publicly available Trusted Execution Environments (TEEs) in CPU, especially Intel SGX [12], a pervasively used TEE product in CPU to shield the offloaded model. SGX provides code integrity and data confidentiality for enclaves, such that the offloaded model shielded in an enclave cannot be seen or tampered with from other untrusted software stack [7, 54].

However, existing SGX-based approaches are facing a fundamental *confidentiality dilemma* between black-box accesses and fast execution: either an approach has to shield all inference computations within enclaves in a CPU to preserve model confidentiality, or outsource part of the sensitive model from CPU enclaves to an untrusted GPU with partially compromised model confidentiality for faster execution. As a typical TEE-shielded work, MLcapsule [29] shields the entire model in enclaves for model confidentiality, but achieves unsatisfactory performance (up to 22.6× higher latency than insecure GPU inference §6.1) for diverse workloads.

In order to mitigate this dilemma, existing work including eNNclave [69] and DeepEnclave [28] take a partition-based approach to leverage the GPU computation power while preserving specific models' confidentiality. In more detail, such an approach requires manually identifying less

sensitive layers with substantial expertise, and using transfer learning [71, 94] techniques to retrain these layers into insensitive public layers. Such an approach can be useful (error-free) when parititioning well-known simple models (e.g., ResNet [79] and VGG [66]) with lucid sensitivity boundaries [78,88]. By partitioning public layers to GPU and shielding other private layers in enclave, sensitive parameters in private layers remain confidential while the untrusted GPU can handle the public layers fastly, achieving faster performance than TEE-shielded systems (e.g., MLcapsule) without compromising confidentiality of these simple models.

Unfortunately, as big models (e.g., Transformer [2]) are getting pervasively used in mission-critical applications deployed at the edge [32,46], existing handcrafted model partition approaches are error-prone to big models and incur significant accuracy drops. This is because big models are often composed of numerous layers with intricate inter-layer linkages [2], making it hard to measure the sensitivity of layers for partitioning, thus a handcrafted partition could potentially compromise model confidentiality (proven in §5.1.2). Moreover, since a difference in single digit of parameters in a well-trained big model would cause severe accuracy drops [35,78], it is impractical to deploy handcrafted model partition approaches on mission-critical applications at the edge that cannot tolerate any accuracy drops.

Our key observation to securely partition general models without losing precision is the *associativity* property of many inference operators. *Associativity* means that the way that factors are grouped in an operator's computation does not change the final result [51]. Since associative operators (e.g., *Convolution*, *Linear*) widely exist in both simple models (e.g., VGG) and big models (e.g., Transformer), hence, we can securely transform a fraction of associative operators into *parameter-morphed* operators to execute in untrusted GPU without revealing the plaintext data, and then recover the execution results in a CPU enclave without losing precision.

Based on this observation, we propose SOTER, the first partition-based approach that provides stronger confidentiality guarantees to general models at the edge. SOTER carries a *Morph Then Recover* (MTR) protocol in enclave to securely partition some inference computations to GPU. Specifically, SOTER profiles the entire model and morphs a fraction of associative operators with randomly generated *blinding coins* (scalars) in enclave, partitions these *parameter-morphed* operators to securely execute in GPU, and fully *recovers* the accurate inference results with reciprocal coins maintained in enclave. With the MTR protocol, all partitioned operators' parameters are hidden without revealing the plaintext while the inference can be accelerated by engaging the GPU power.

Even with the MTR protocol, SOTER still faces an integrity threat in the presence of a third-party attacker at the edge. Specifically, the client device might be hijacked by a third-party attacker who can observe and manipulate any components outside enclave (e.g., morphed parameters in GPU), misleading a model to produce wrong outputs to drop the inference accuracy and causing application failures. Such an integrity threat is a general open problem in existing partition-based approaches [28,69] and none of these work tackles such an integrity threat.

To tackle this open integrity challenge, SOTER develops an *oblivious fingerprint* technique to detect integrity breaches outside enclave in a *challenge-proof* manner [92]. Specifically, before the inference, SOTER collects several fingerprints of partitioned operators which hold the ground-truth correctness of these operators' execution results. Then, during the inference, SOTER randomly challenges partitioned operators to ask them for execution proofs. By comparing the returned proofs with expected fingerprint outputs in enclave, SOTER can learn whether an integrity breach happens.

To prevent an adversary distinguishing fixed fingerprint challenges and bypassing integrity detection, stemming from the same associativity observation, SOTER dynamically derives new fingerprints at runtime that statistically share the same distribution as the normal input to partitioned operators (§5.2), making fingerprints oblivious to the adversary. Therefore, by leveraging the same key observation, we can achieve both model confidentiality and integrity in an unified system.

We implemented our SOTER system on Slalom [80] codebase, a mature and lightweight framework for writing neural networks. We evaluated SOTER with nine state-of-the-art DNN models covering four prevalent types: Multi-layer Perception (MLP), Convolution Neural Network (CNN), Recurrent Neural Network (RNN) and Transformer. We compared SOTER to two notable TEE-based client-side inference systems, including the TEE-shielded approach MLcapsule [29] and partition-based approach eNNclave [69]. Our evaluation results show that:

- **Security:** In terms of a widely used metric for evaluating model confidentiality [85, 95], SOTER achieved comparable confidentiality guarantees compared to TEE-shielded approach (i.e., MLcapsule [29]), and achieved 24.8% stronger confidentiality on average for big models than partition-based approach (i.e., eNNclave [69]) (§5.1.2). For integrity, SOTER quickly detected integrity breaches with a high probability of 99.39% (§5.2), while all existing partition-based approaches [28,69] are not designed to tackle such vulnerability.
- **Performance:** SOTER was efficient. SOTER achieved up to 22.6× lower inference latency than the TEE-shielded approach on big models, and had comparable latency compared to the partition-based approach (i.e., eNNclave) (§6.1).
- **Accuracy:** SOTER was accurate. SOTER achieved the ground-truth accuracy same as insecure inference, while existing partition-based approach (i.e., eNNclave) caused 1.1%∼6.3% accuracy drops (§6.2).

Our major contribution is the MTR protocol, the first work that automatically provides stronger confidentiality and integrity guarantees to general DNN models' inference at the

edge. Compared to existing relevant systems, SOTER's security gains are two-fold: for confidentiality, SOTER achieved the same guarantee as the handcrafted partition-based systems for all their evaluated specific models, and SOTER provides stronger guarantees for general and complex big models (see Figure 6 in §5.1.2), which are infeasible to handcraft in these exiting systems; for integrity, SOTER effectively detected integrity attacks with overwhelming high probability (99.39%, proved in §5.2), while all existing partition-based systems are not designed to handle such integrity attacks.

Furthermore, our extensive evaluation on prevalent models shows that SOTER achieves reasonable performance overhead, which is comparable with handcrafted systems and much better than existing TEE-shielding systems; SOTER also achieves high accuracy, which is the same as insecure inference and much higher than handcrafted systems. This makes SOTER unique to greatly promote general security-critical AI applications and to effectively protect diverse well-trained proprietary DNN models at the edge, including home monitoring [18], visual assistance [59] and autopilot navigation [8]. SOTER's source code is released on `github.com/ndss22p16/soter`.

In the remaining of this paper, §2 introduces background. §3 gives an overview of SOTER. §4 describes SOTER's design and security analysis. §6 shows our evaluation. §7 discusses related work and §8 concludes.

2 Background

2.1 Deep Neural Network

A DNN model (in short, model) can be represented as a sequence of connected *layers* with each layer assigned a set of *operators*, as shown in Figure 1. An operator is either a *linear operator* or a *nonlinear operator* where a linear operator is weighted by *parameter matrices* (in short, parameters).

Inference workflow. During the inference phase, a model takes input data and passes the data through all operators for computing a final output. Without losing generality, we use image classification as an example to illustrate the model composition in next discussions.

Figure 1 shows the inference workflow. A model M passes an input X (e.g., an image) through layers of operators to compute *logits* [31], normalizes logits with *softmax* function to produce a probability vector, and assigns a class with the highest probability to input X as the class label.

Associativity property of DNN operators. Operators are the basic building blocks of a DNN model where linear operators have been proven to take up the most computation resources in inferences [3, 91].

Enormous DNN models deployed at the edge are built on top of *associative* operators. Suppose we have an input X and a scalar μ, a DNN operator F is associative if

$$(\mu^{-1}\mu)F(X) = \mu^{-1}F(\mu X) \tag{1}$$

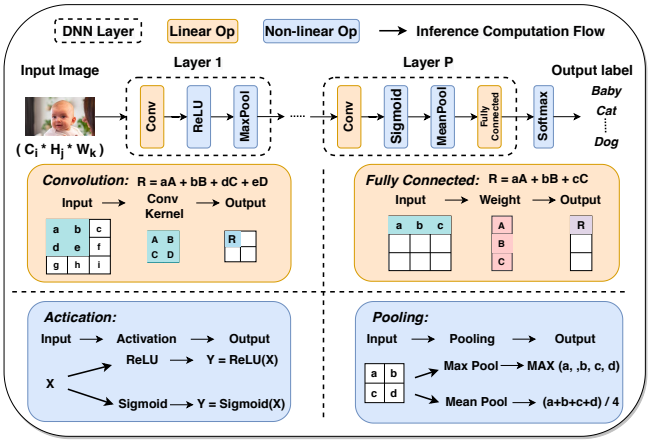


Figure 1: An overview of a DNN model architecture.

Linear operators including *convolution* and *fully-connected* all have the associativity property as they conduct linear transformations on an input data. Take the most commonly used convolution operator [74] as an example. As shown in Figure 1, if we multiply each element in the convolution kernel by 2^{-1} (i.e., $\mu = 2^{-1}$), we will get the output $2^{-1}R$ while it always holds $(2*2^{-1})*R = 2*(2^{-1}R)$. This property is applicable to the *fully-connected* operator as well.

For nonlinear operators that conduct nonlinear transformations on an input data, most of them do not have an associativity property (e.g., *Sigmoid*). However, under specific constraints, some nonlinear operators can also be associative. Specifically, when $\mu > 0$, the *ReLU* operator is associative because μ will not change the positive or negative of the input X, such that $\mu^{-1}max\{0, \mu X\}$ will get the correct output. The same constraint is applicable to the *max-pool* operator as well. Note that the *mean-pool* operator naturally has the associativity property because it actually conducts linear transformation (i.e., averaging) on an input data.

Operators that satisfy Equation 1 also meets a variant of the associativity property. Given that

$$F(X_1) = Y_1 \quad | \quad F(X_2) = Y_2$$

we have

$$F(\mu_1 X_1 + \mu_2 X_2) = \mu_1 Y_1 + \mu_2 Y_2 \tag{2}$$

SOTER leverages Equation (1) and Equation (2) as the key weapon to automatically partition general models without requiring to manually identify confidentiality-sensitive operators in a handcrafted manner, which is error-prone to general complex models. We illustrate our detailed designs in §4.

2.2 Intel SGX

Intel Software Guard eXtension (SGX) [12] is a pervasively used hardware feature on commodity CPUs. SGX provides a secure execution environment called enclave, where data and code execution cannot be seen or tampered with from outside. Code outside enclaves can enter an enclave by `ECalls`.

There are two categories of work that leverage SGX to protect model confidentiality when running inference at the edge. The first category is the TEE-shielded approach that runs all inference computations within TEE (e.g., MLcapsule [29]). Such an approach shields all model information (both the architecture and parameters) in a trusted enclave but fails to achieve low inference latency owing to the computational bottleneck of the CPU.

The second category is the partition-based approach, including two notable systems eNNclave [69] and DeepEnclave [28]. Existing partition-based approaches use either transfer learning techniques [71] or knowledge distillation [11, 90] to manually partition some layers running in a co-located untrusted GPU (more discussion in §7.1), which cannot support general complex big models (e.g., NASNet [94], Transformer [2]). Meanwhile, exiting partition-based approaches do not handle the inherent integrity threat after model partitioning, exposing a large attack surface to random perturbation attacks [6,37,58,60] to partitioned layers outside enclave.

3 Overview

3.1 System setup

We consider a client-side inference scenario shown in Figure 2. The client owns sensitive inference queries for a specific AI task. The server owns a well-trained model for the task. To run the client-side inference, the server offloads its model to the client's TEE-equipped edge device. The device constantly takes sensitive queries from the client and sends inference results back to the client.

3.2 Threat model and privacy goals

Similar to other client-side inference systems, namely the MLcapsule [29], eNNclave [69] and DeepEnclave [28], we adopt a semi-honest threat model, where both the client and server have a bounded computational capability [93, 95], and try to learn information about the other party's private data from the messages it receives. That is, the client C and server S will follow the inference protocol but C wants to learn S's proprietary model architecture and parameters, and S attempts to learn C's input data. Since the two parties have conflicting interests, we exclude collusion attacks here.

Different from prior work that assumes the offloaded model is built on well-known DNN architectures such as ResNet [79] and VGG [10, 66], we intend to protect both model parameters and architecture (number of layers, linkages between layers, etc) because recent advances in AI has demonstrated that model architecture has a significant impact on achieving high inference accuracy [4, 9, 19], making a well-designed architecture proprietary to get protected.

Note that, we do not hide information that is revealed by the results of inference queries, and we focus just on protecting

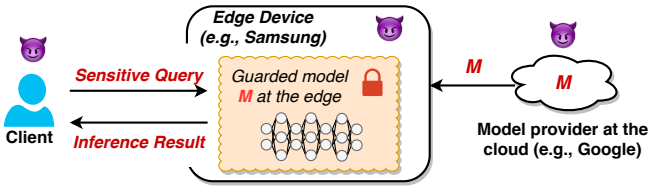


Figure 2: The client-side model inference scenario.

clients' queries and the offloaded model during the inference process. This assumption stays in line with the threat model of all existing black-box inference systems [29, 38, 53, 69]. Protecting against such attacks that exploit the leakage of inference results is a complementary problem to that solved by SOTER. We give detailed illustration of these attacks and potential mitigations in §7.2.

SGX's threat model. We use the Intel SGX enclave [12] as the TEE in our work because SGX has been pervasively used in existing secure inference systems [29,69] and is commonly available in modern Intel CPUs [12]. But in principle, our approach can also be generalized to other TEEs [40,63] with data confidentiality and code integrity protections.

SOTER has the same threat model for SGX as typical SGX-based systems [36,73,82]. We trust the hardware and firmware of Intel SGX, which ensure that code and data in enclave can not be seen or tampered with from outside. Same as eNNclave [69] and other SGX-based secure inference systems [28, 29], we do not intend to address the vulnerabilities of side-channel attacks in this paper, but we still discuss potential vulnerabilities and mitigations in §7.2.

Integrity threat model. We consider a third-party attacker can hack into the client device, observe and manipulate any components outside enclave (e.g., parameters exposed in GPU) to cause integrity breaches during the inference process. Such an attack widely exists in an edge computing ecology [26, 41, 48]. For instance, an attacker may hack into an autopilot system, perturb the obstacle prediction process (i.e., inference) to produce wrong navigation instructions [68].

3.3 System overview

SOTER's two-phase design. Figure 3 shows the architecture of a SOTER client. At a high level, SOTER has three trusted modules (in green) running in two phases. Specifically, the *profiler* module runs in an offline *preprocessing phase*, which loads an offloaded model and automatically makes a confidential partition plan; the *inference manager* module and *integrity monitor* module run in an online *inference phase*. These two modules execute normal inference queries at runtime and dynamically derives oblivious fingerprints to detect integrity breaches of partitioned operators outside enclave.

Note that, SOTER's two-phase design is most suitable for inference applications [24, 25] whose inference is latency-sensitive, but is usually not performed frequently enough to take up all computational resources needed for preprocessing.

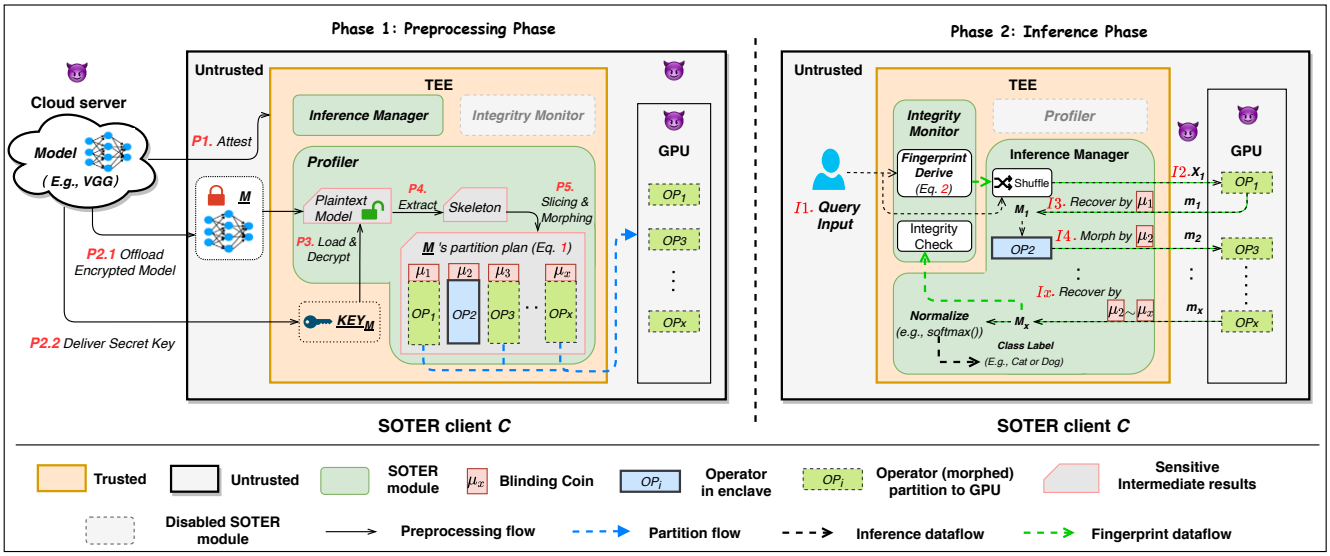


Figure 3: The architecture of SOTER in a modular manner. SOTER's secure inference protocol has three modules (shielded in green) running in the enclave. An adversary can observe and arbitrarily modify parameters of partitioned operators in GPU.

SOTER's workflow. Figure 3 shows the workflow of SOTER. Next, we introduce the workflow and highlight the challenges behind our major decision choices.

Preprocessing workflow. During **P1**∼**P3**, a SOTER client conducts standard SGX attestation to the server for obtaining the model M and decryption keys KEY_M, and then loads the encrypted model M in a layer-wise manner by decrypting with KEY_M. In **P4** and **P5**, SOTER extracts the model architecture and then automatically makes a confidential partition plan by utilizing the general *associativity* property of operators.

Specifically, in **P5**, SOTER automatically partitions a model as follows. With a given security parameter θ, SOTER randomly selects θ% associative operators and produces (1) random positive scalars for *all* operators, and (2) a set of *fingerprints* of selected operators for integrity checking. These scalars serve as the *blinding coins* to hide the operators' parameters [53] and are always kept secrete in enclave. Owing to the associativity property (Equation 1), SOTER morphs every selected operator by multiplying with a coin (the result can be accurately recovered with the reciprocal coin), and then partitions morphed operators to GPU (§4.1).

Preprocessing highlight. A critical challenge is to choose an appropriate θ in the preprocessing phase. Even if the exact value of partitioned operators' parameters are hidden, the more operators are partitioned (with a larger θ), the more confidentiality loss would potentially incur under adversarial training attacks [85, 95] and parameter-deriving attacks [60]. Hence we must choose an appropriate θ to minimize the confidentiality loss and maximize the performance gain (§5.1).

Inference workflow. During the inference, there are two types of data flows: the inference flow (i.e., client queries in black dotted line) and the fingerprint data flow for integrity checking (green dotted line). In **I1**∼**I4**, the client's query input (e.g., an image vector) is passed through all operators and computed in a *Morph Then Recover* way whenever a cross kernel (TEE-GPU) transmission happens. Specifically, (**I1**) the inference manager module takes client C's input X_1 and shuffles with fingerprints (introduced below). (**I2**) The inference manager forwards X_1 to the first *morphed* operator OP_1 and computes the output m_1. (**I3**) m_1 is returned to OP_2 in enclave, the inference manager takes m_1 as input, recovers m_1 with all previous partitioned operators' coins since the last kernel switch (in Figure 3, SOTER recovers with only OP_1's coin μ_1), and gets the recovered result M_1. (**I4**) After that, OP_2 in enclave computes M_1, gets an intermediate result (IR), then morphs IR with the OP_2's coin μ_2 and forwards the result m_2 to the partitioned operator OP_3 in GPU (§4.1). The above procedures (**I2**∼**I4**) repeat until all partitioned operators in GPU are computed. In (**Ix**), we recover the final results with all previous partitioned operators' coins since the last kernel switch. Then SOTER runs a standard DNN normalization function (e.g., *softmax()*) get the final class label in enclave.

The fingerprint data flow is used to detect integrity breaches of partitioned operators. Specifically, the integrity monitor module dynamically derives a set of fingerprints (having ground-truth execution proofs of partitioned operators) by utilizing the general associativity of Equation 2. SOTER randomly challenges partitioned operators for execution proofs, and compares the proofs with expected fingerprint output. Any mismatch reveals integrity breaches outside enclave (§4.2).

Obliviousness challenge. Unfortunately, trivially using fingerprints is not secure for detection because the adversary can continuously monitor the inference process and observe every IR transmitted from enclave to GPU. If using only a fixed set of fingerprints to challenge partitioned operators, the adversary can easily distinguish those fixed fingerprints and

bypass detection by returning correct proofs (§5.2). SOTER tackles this problem by making fingerprints oblivious to the adversary while ensuring timely detection by issuing only a few fingerprint challenges (§4.2).

4 Protocol Description

This section describes SOTER's MTR protocol for partitioning general DNN models with confidentiality and integrity guarantees. At a high level, MTR utilizes the general associativity property of many DNN operators to automatically profile the entire model (with Equation 1), randomly selects θ% ready-to-partition operators to hide the model architecture, and then morphs these operators' parameters with hidden *blinding coins* in enclave to hide the plaintext data (§4.1).

Besides, to tackle the inherent integrity threat of inference manipulations from a third-party attacker (described in §3.2), stemming from the same observation of the general associativity property, MTR dynamically derives *oblivious fingerprints* (with Equation 2) and uses fingerprints in enclave to check operation integrity in GPU in a *challenge-proof* manner (§4.2).

We now give the formal definition of a client-side partition-based secure inference protocol as follows.

Definition I. *An inference protocol* Ψ *running at client C holds an offloaded model M with architecture* α *and parameters* $(M_1,...,M_l)$*, and the client C holds input queries Q to run inference on M.* Ψ *is a highly-desirable secure inference protocol at the client side if it satisfies the following rules:*

- ***Security:***

- ***Corrupted server.*** *We require that a corrupted, semi-honest server does not learn anything about the input queries Q of the client C.*

- ***Corrupted client.*** *We require that a corrupted, semi-honest client C does not learn anything about either the model M's architecture* α *or the plaintext parameters* $(M_1,...,M_l)$*. Formally, with a partitioned model* M_p *under the view of C, we require that (1)* Ψ *should guarantee a low probability to derive the true parameters* $(M_1,...,M_l)$*, and (2)* Ψ *should guarantee a low transferability of* M_p*, which is defined as the ratio of the correctly computed inference queries that the partitioned model* M_p *can mimic the original model M (see §5.1.2).*

- ***Hijacked client.*** *Given that a client C might be hijacked by a third-party attacker, who can observe and manipulate the inference process to harm the service, we require that* Ψ *has to quickly detect integrity breaches outside enclave with overwhelming high probability and zero false positives.*

- ***Performance:***

- ***Low latency.*** *We require that without sacrificing security,* Ψ *should partition as many computations as possible to a co-located accelerator (e.g., GPU) for speeding up inference, given that client-side AI applications always have stringent latency requirements.*

Table 1: SOTER's protocol variables.

Variable	Description
$\Theta(default = 0.8)$	Portion of associative operators partitioned to GPU.
$O_g \mid O_e$	Partitioned operators in GPU / maintained in TEE.
$O(O_g \cup O_e)$	The whole set of model M's operators.
$Para(O_i)$	Parameters of operator O_i.
FP_{O_i}	Fingeprint of operator O_i.
μ_i	Blinding coin of operator O_i.

- ***High accuracy.*** *Given a partitioned model* M_p*, on every input query Q from client C, we require that the output* $M_p(Q)$ *returned by* Ψ *should achieve the same inference result as the original model* $M(Q)$ *without any accuracy loss.*

4.1 Morph Then Recover (MTR) protocol

The MTR protocol is divided into two stages: a *morphing* stage in the preprocessing phase (handled by the profiler module), and a *recover* stage in the inference phase (handled by the inference manager module). In the morphing stage, SOTER first makes *blinding coins* for every operators (including both associative and non-associative operators), and then makes a partition plan with a given security parameter θ. Then, in the recover stage, SOTER runs inference across GPU and enclave, and recovers inference results in enclave when needed. Algorithm 1 and 2 show the MTR protocol.

Stage 1-1: Morphing with blinding coins. SOTER assigns every operator with a randomly generated scalar, which serves as the blinding coin to hide the plaintext value of parameters. Given an operator O_i (the partition plan is described in **Stage 1-2**), SOTER morphs $Para(O_i)$ by multiplying each element in $Para(O_i)$ with the corresponding coin μ_i.

The blinding coin design protects model confidentiality from two aspects. First, by hiding coins within enclave, we intuitively hide the plaintext data of real parameters with scaled (morphed) parameters.

Prevent leakage during kernel switches. Second, we assign every operator (rather than every ready-to-partition operator) with coin to avoid potential information leakage during kernel switches, i.e., when intermediate results (IRs) are transmitted between enclave and GPU. Consider a case in Figure 3 where OP_1 (in GPU) is a linear operator and OP_2 (in enclave) is a non-linear *ReLU* operator. If we only morph OP_1 with μ_1 and do not assign a coin to OP_2, then, with a client input X, OP_1 outputs $y_1 = X(\mu_1 * OP_1)$ and OP_2 outputs $y_2 = ReLU(y_1/\mu_1) = ReLU(X * OP_1)$. However, since the *ReLU* operator just filters out negative values in parameter matrices and does not transform a scalar, an adversary who observes both y_1 and y_2 will directly infer the value of coin μ_1, violating the confidentiality of the partitioned OP_1 (concretely, $Para(O_i)$). To tackle this problem, we assign coins to all operators rather than partitioned operators only. By assigning coin μ_2 to OP_2, OP_2 will output $y_2 = \mu_2 * ReLU(X * OP_1)$. An adversary observes $\mu_1 * \mu_2$ but cannot infer either μ_1 or μ_2.

Algorithm 1: MTR protocol at client c (offline) (§4.1).

```
▽ Preprocessing Phase (within enclave).
1  Function partition(O) do
2      foreach operator op in O do
3          op.partition ← False; op.index ← O.index(op);
4          if op.index > W then
5              if 0 < normalize(sgx_read_rand()) < Θ then
6                  op.partition ← True;
7              if op is associative & op.partition then
8                  foreach element e in para(op) do
9                      e ← μ_index × e ;              // Morph
10                 O_g.add(op) ;                  // GPU operators
11             else
12                 O_e.add(op) ;                  // TEE operators
13     O_g.copyTo ("cuda") ;                  // Partition to GPU
```

Stage 1-2: Partitioning with hidden architecture. With all coins prepared, given a security parameter θ, we automatically partition a portion of associative operators (that are morphed) to GPU for inference acceleration. The *partition* function in line 1∼13 of Algorithm 1 shows the pseudo code.

Specifically, in the preprocessing phase, SOTER iterates every operator in model M and randomly selects a portion of associative operators and adds them to the partition set O_g. For instance, with $\theta = 0.8$, an associative operator would have an 80% chance to be partitioned to GPU. After all operators are iterated, all operators in O_g are partitioned to GPU for inference acceleration.

Note that, SOTER always keeps top-W (W=2 by default) operators in enclave even if these operators are associative (Line 4). This design choice is made to ensure the input to the first several partitioned operators in GPU (e.g., X_1 in Figure 3) are always unknown to the adversary, such that we can stealthily check the integrity of all partitioned operators by injecting fingerprint challenges (more details in §4.2).

Figure 4 depicts an example of running the MTR protocol to automatically partition general neural networks. To illustrate our protocol in a lucid manner, in this example we choose $\theta = 1$, which means that all associative operators will be partitioned to GPU for acceleration.

In the above example, for VGG-19, we can partition most of its operators to GPU because VGG-19 is a typical layered model composed mostly by associative operators including *convolution*, *ReLU* and *max-pool*, given that the last two non-linear operators can also meet the associativity property with a positive coin (§2.1). Besides, most non-associative operators in VGG-19 such as *BatchNormal2d* and *DropOut* are automatically de-activated during the inference process to avoid non-determinism [72]; For WaveRNN, we give partition plans for its encoding (decoding) layer as WaveRNN is built by iteratively stacking these two layers and other scattered layers. Different from VGG-19, as a typical RNN model, WaveRNN has more non-associative operators (e.g., *Sigmoid*) that cannot be partitioned to GPU for acceleration.

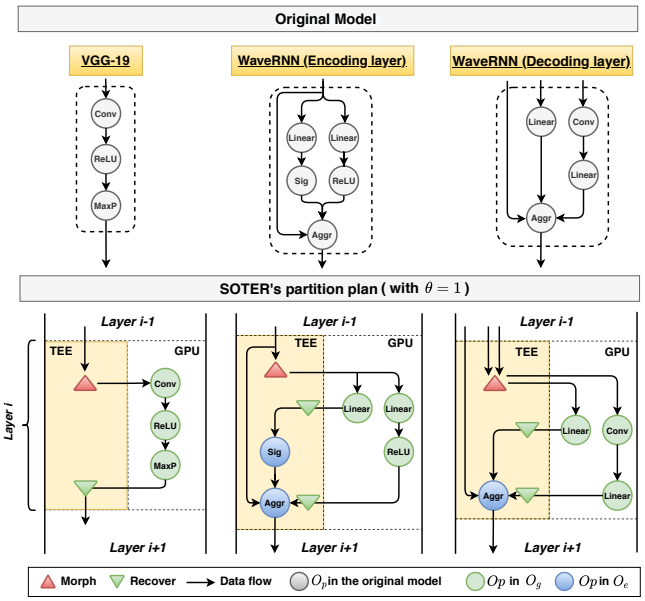


Figure 4: Examples of running SOTER's MTR protocol to partition two typical DNN models.

Stage 2: Guarded black-box inference. Next we present the end-to-end black-box inference process given that we have prepared coins and partitioned a portion of operators to GPU.

As shown in Algorithm 2, SOTER first computes the reserved top-W operators in enclave, and then iterates **I2**∼**I4** as follows. **I2**: SOTER finds the longest length of consecutively partitioned operators in O_g and computes the inference result by forwarding the input through these operators (line 6∼11). **I3**: Then SOTER copies the result back to enclave, recovers the real inference result by multiplying the reciprocal coins of all operators in last step (line 13), and then forwards the result to operators maintained in enclave (before the next copy to GPU). **I4**: SOTER morphs the forwarding result with all coins of enclave operators in last step (line 18), and then copies back to GPU. This procedure terminates when all partitioned operators in GPU are iterated. Then SOTER normalizes the final result and returns the result to the client.

4.2 Integrity checking with fingerprints

Obliviousness requirement. To detect integrity breaches outside enclave (as described in §3.2), a straw man approach could be using a fingerprint (with ground-truth input/output pair) to *challenge* the partitioned operator in GPU to recompute the fingerprint input to provide a *proof*, and report a crime if the proof is different from the expected output.

However, such an approach needs to be oblivious. Since the adversary can continuously monitor the inference process and observe every intermediate result (IR) transmitted from enclave to GPU, if using only a fixed set of fingerprints, the adversary can easily distinguish those fixed challenges among all IRs, and bypass our detection by returning correct proofs.

Algorithm 2: MTR protocol at client c (online) (§4.1).

```
   ▽ Inference Phase (cross enclave & GPU).
1  Function secure_inference() do
       ▽ I1.
2      every client input X do
3          X.copyTo("enclave");
4          foreach index i ∈ normalize(W) do
5              X ← O_i(X) ;                    // Top-W Inference
6          start ← next op in O_g; end ← next op in O_e;
7          while start < |O| do
               ▽ I2.
8              X.copyTo("cuda");
9              foreach index i ∈ range(start, end) do
10                 X ← O_i(X) ;                // GPU Inference
               ▽ I3.
11             X.copyTo ("enclave");
12             X ← ∏_{start}^{end-1} μ_i^{-1} × X ;   // Recover
13             start ← end;
14             end ← start from O_end find the next op in O_g;
15             foreach index i ∈ range(start, end − 1) do
16                 X ← O_i(X) ;                // TEE Inference
               ▽ I4.
17             X ← ∏_{start}^{end-1} μ_i × X ;        // Morph
18         X ← normalize(X);
19         return Top_1 (X) ;                  // Final result
```

The timeliness-obliviousness dilemma. Unfortunately, trivially using different fingerprints for integrity checking brings a *timeliness-obliviousness* dilemma.

Specifically, generating new fingerprint input and pre-computing expected output in CPU would cause late detection, because CPU computation for an operator is extremely slow (up to 30x slower inference speed for linear operation); on the other hand, if we use a fixed set of fingerprints for challenging the integrity, we could timely detect breaches. However, the detection would be easily bypassed by the adversary because all fingerprints are now distinguishable.

SOTER's insight and approach. Stemming from the same observation of parameter morphing (§5.1), we tackle this challenge by efficiently generating new fingerprints in CPU without re-computing operators. At a high-level, we take two steps by first making a set of *cornerstone fingerprints* for each partitioned operator, and then encapsulate new fingerprints from these cornerstone fingerprints with linear cost at runtime based on the associativity property.

Step 1: Preparing cornerstone fingerprints. During the pre-processing phase (line 18∼19 of Algorithm 1), SOTER prepares K (by default K=10) cornerstone fingerprints for each partitioned operator. A fingerprint is a two-element tuple: $\{input, output\}$, where the input is a randomly generated matrix and the output is pre-computed by forwarding the input through a ready-to-partition operator. The whole preparing procedure is running within enclave, such that the correctness of each cornerstone fingerprint can be guaranteed.

Step 2: Efficiently deriving new fingerprints in TEE. Now with K cornerstone fingerprints for each partitioned operator, SOTER efficiently derives oblivious fingerprints at runtime by leveraging the general associativity property. Specifically, with a fixed set of cornerstone fingerprints $FP_O[K]$ for each partitioned operator O, we randomly selects T fingerprints and generates T random coefficients with SGX's trustworthy random number generator. Then, by applying the associativity property (Equation 2), we generate a new fingerprint FP_{New} as follows: the input for FP_{New} is $\sum_{i=1}^{T} T_i * FP_O[i].input$, i.e., we multiply each selected cornerstone fingerprint's input with a corresponding coefficient and add them all as the new input; For the output, since the associativity property implies that the corresponding output for a group of associated operators has the same transformation as the input, thus we can directly get the expected output as $\sum_{i=1}^{T} T_i * FP_O[i].output$ without conducting slow CPU inference for a given new input.

Step 3: Challenge-proof fingerprint issuing. To detect any integrity breaches on partitioned operators with overwhelming high probability (see **Remark I**), we issue a *challenge* with probability p whenever a kernel switch (an IR transmitted from enclave to GPU) happens. In more detail, when an IR (produced by client's query) is sent to a partitioned operator O, we challenge the integrity of O by sending with a new $FP_O.input$ (produced in step 2), and compare the returned *proof* with the expected $FP_O.output$. Any mismatch reveals an integrity breach of partitioned operator O.

Remark I (Detection probability and performance impact). The challange probability p affects both the detection rate and performance: adding p will increase the detection probability and timeliness, but comes at a runtime overhead to GPU. We prove (and evaluate) that SOTER can achieve overwhelming high detection probability (§5.2) and reasonable performance overhead (§6) with a proper choice of p.

5 Analysis of SOTER Security Properties

In this section, we present the analytical and experimental evaluation on the security of SOTER. We first demonstrate that our MTR protocol provides sufficient query confidentiality and model confidentiality during the inference process. Then, we analyze the integrity under random perturbation attacks, and show that our solution can handle such cases gracefully.

5.1 Confidentiality Analysis

5.1.1 Corrupted Server

Attacker 1. We consider a malicious server S who wants to abuse client's input queries X. Such vulnerability widely exists in AI-driven applications. For instance, in face recognition, S might maliciously collect client's face images to strengthen (train) its own model [22].

Intuitively, such an attacker is naturally warded off by the design of SOTER. This is because all communication between

client and server happens before the queries being observed by the server. As the client makes inference offline and the results were generated within the trusted enclave (shown in Figure 3), we ensure that the input queries will never get leaked and a malicious server cannot break the query confidentiality.

5.1.2 Corrupted Client

Attacker 2. We consider a semi-honest (i.e., honest but curious) client who follows our MTR protocol coded within SGX but wants to train a *substitute model* with several observations of partitioned model architecture and parameters outside enclave. Such adversary could be a business competitor whose purpose is to steal the offloaded model, in order to reduce the competitive advantages of the model owner.

We emphasize two assumptions ahead of the attack analysis here. First, we assume the adversary has a *bounded computational capability* as in [60, 85], and does not know the whole training dataset. Otherwise, it is totally unnecessary for the adversary to "steal" the model. This assumption stays in line with existing partition-based inference systems to evaluate the confidentiality guarantee of a model's partition plan [85, 95].

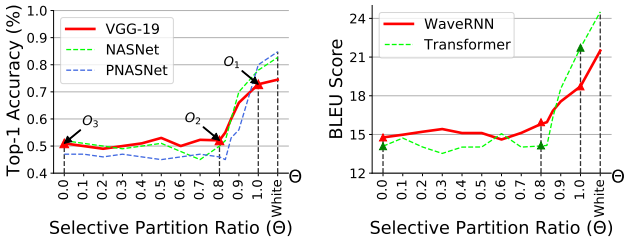
Second, we do not consider any black-box attacks here and we assume client's queries are *benign*. While a corrupted client might use SOTER as a black-box oracle to extract the model or even the training dataset [49, 60, 61, 86], SOTER does not aim to protect against such black-box attacks. Instead, SOTER focuses just on protecting the offloaded proprietary model during the inference process (more discussion in §7.2).

Evaluation Metric. Due to the heuristic nature of DNN models [20], similar to prior work [85, 95], we give an experimental security analysis to evaluate the model confidentiality of our MTR protocol with the *transferability* metric.

Transferability means the portion of functionalities that a substitute model *SM* (trained by the adversary) can mimic the original offloaded model *M* [85]. A higher transferability indicates that the *SM* trained by the adversary can mimic a larger portion of functionalities (i.e., more confidentiality loss), while a lower transferability reveals that a smaller portion have been stolen (i.e., less confidentiality loss). For CNN models, we use the *Top-1* accuracy as the transferability metric and we use the *BLEU score* for the RNN models, both of which are widely-adopted model measuring metrics [64].

Substitute training setup. We experimented the adversarial substitute training (*Attack 2*) with five prevalent DNN models, pre-trained them with CIFAR-10 dataset [42] (for CNN) and WMT16 EN-DE dataset [70] (for RNN and transformer), respectively. With the bounded computational capability assumption, to stay in line with [60, 95], we isolated 90% of training data (e.g., 45,000 images in CIFAR-10), and the remaining 10% of training samples are used (and augmented with [60]) by the adversary.

Following state-of-the-art substitute training setup [85], to construct the substitute model, we adopted VGG-13 as the backbone network for CNN models, and we used transformer (with encoder-decoder architecture) as the default substitute model for NLP models. With the substitute model, for operators having the same size as the observed target operators (with morphed parameters) outside enclave, we simulated the adversary by fitting these observed target operators into the substitute models; for other operators, we initialized all parameters using a truncated normal distribution with std of 0.02. Same as [85], we used Adam to train the substitute model with 0.0001 learning rate. By training with one Nvidia 2080TI GPU, the CNN models gradually converged between 80∼150 epochs while NLP models converged within 10 epochs.



(a) CNN models (b) NLP models

Figure 5: SOTER's θ-Transferability analysis of prevalent DNN models on the adversarial substitute model $SM(\theta)$.

θ-Transferability analysis. We now analyze the model confidentiality of our MTR protocol. In particular, we give experimental results on the effects that the selective partition ratio θ could have on the quality of the adversarial substitute model *SM* trained by the adversary. With a given θ, we denote the corresponding substitute model as SM_θ.

To give analysis in a lucid manner, we first analyze the transferability of SM_θ when the target model is a typical CNN model VGG-19, and then we give analysis on other prevalent DNN models. Specifically, for VGG-19, we simulated an adversary who trains different *SM*s based on the following partial views outside enclave, as shown in Figure 5a.

- *Non-shielded model.* The red line (VGG-19) can achieve the ground-truth Top-1 accuracy (74.5%) when the X-axis is "*White*", i.e., a scenario where the entire model was entirely white-box without activating our MTR protocol (i.e., SM_{White}). The ground-truth accuracy result is intuitive due to the model outside enclave being the same as the original model.
- *TEE-shielded model.* When the X-axis $= 0$, which means no operators were partitioned to GPU (i.e., fully shielded in enclave), the adversarial substitute model SM_0 achieved 54.3% accuracy, showing a low transferability of the target model *M*. This also indicates that even with a TEE-shielded model, the adversary can still train a substitute model from a blank model with a partial dataset through queries.
- SOTER *Morphed model.* When the selective partition ratio θ downgrades from 1 to 0, we observed a significant accuracy (transferability) drop from 71.8% to 54.3%, which shows the effectiveness of our protocol of using blinding coins to hide parameters. Besides, we emphasize three key points (**O1**∼**O3**

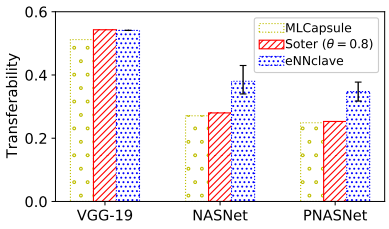
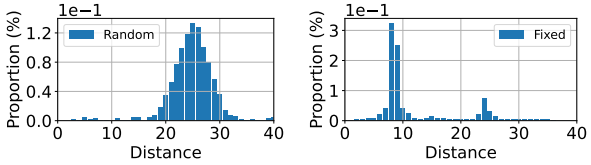


Figure 6: Comparison of transferability between SOTER and other baseline systems. A lower transferability (Y-Axis) indicates a stronger confidentiality guarantee on resisting adversarial substitute training (***Attacker 2***) on that model (X-Axis).



(a) Oblivious fingerprint. (b) Fixed fingerprint.

Figure 7: The l_2 distance distribution of randomly sampled IRs with oblivious or fixed fingerprints.

in Figure 5a) from our experiments.

First (**O1**), when $\theta = 1$, SM_1's accuracy decreases $\sim$2.7% because in that case the adversary can train SM_1 with the knowledge of the major architecture of model M (since operators used in VGG-19 are mostly associate, the adversary will get most of the operators' information when $\theta = 1$). Hence, the adversary could achieve a resultant substitute model SM_1 with a relative high accuracy (i.e., high transferability).

Second (**O2**), when SOTER chooses $\theta = 0.8$, we observed a significant accuracy drop to 54.7%. This is the scenario where SOTER partitions any associate operators to GPU with 0.8 probability. In that case, we hide the architecture information of the model, and the adversary can only train $SM_{0.8}$ with fragmented operators' information and morphed parameters of only a fraction of partitioned associative operators. As a result, the adversary could only train a low-accuracy model $SM_{0.8}$, which cannot substitute the original high-accuracy model M because of the low transferability.

Last (**O3**), when the selective partition ratio was within the range of 0$\sim$0.8, we observed that the accuracy was almost unchanged. This is because, even with a smaller θ, which means less associative operators were partitioned to GPU, the difficulty for the adversary to train a substitute model SM_θ remained almost unchanged.

Based on these results, we choose $\theta = 0.8$ in SOTER by default, which obtains the maximum performance benefit (as most associate operators would be partitioned to GPU for fast processing) when achieving a comparable confidentiality guarantee as black-box models (i.e., $\theta = 0$). We also extended the transferability analysis to NASNet, PNASNet, WaveRNN and Transformer, shown in Figure 5a and Figure 5b. The similar downgrading trend and θ-*Transferability* relationship revealed that hiding with 80% morphed information could be enough to guarantee sufficient model confidentiality.

Baseline comparisons. MLcapsule [29] is a pure TEE-shielded inference system, which prohibitively incurs high inference latency but achieves the strongest model confidentiality because all operators are shielded within trusted enclave (corresponds to the $\theta = 0$ scenario in SOTER). As shown in Figure 6, in terms of the transferability, SOTER ($\theta = 0.8$) could achieve comparable confidentiality as MLcapsule.

Another notable system eNNclave [69] takes a handcrafted approach to manually identify insensitive layers, fine-tune these layers' parameters and directly partition them to run in GPU (§7.1), which requires substantial expertise and usually causes accuracy drops [69]. As eNNclave does not mention any strategies in its paper for partitioning general models other than VGG-series, to fairly compare with it, we randomly partitioned complex models and recorded transferability fluctuations as error bars, shown in Figure 6. eNNclave only provided sufficient confidentiality (low transferability) to VGG model, because protecting several specific DNN layers [69] could be enough to achieve strong confidentiality in simple models, which is not applicable to general complex models, proved by the much higher transferability than SOTER and MLcapsule.

Overall, we analyzed our MTR protocol and we experimentally prove that: SOTER's parameter morphing is effective to protect model confidentiality (see Figure 5); SOTER's confidentiality guaranteee is comparable to the strongest TEE-shielded approach and is much stronger than existing handcrafted partition-based approach(see Figure 6).

Attacker 3. We consider a semi-honest client who continuously monitors the inference process and collects a set of morphed operators outside enclave, aiming to derive the scalars or true parameters.

We demonstrate our security by proving the low probability of successfully launching such an attack. Specifically, even the adversary observes N morphed parameters ($mp_0 \sim mp_N$) hidden with scalars ($\mu_o \sim \mu_N$), the adversary still has a low probability to derive the true $Para(O)$. This is because a parameter matrix can continue to do product: $Para(O) = \alpha * Para'(O)$ while $mp_i = \mu_i * Para(O) = \mu_i * \alpha * Para'(O) = \mu_i * \alpha' * Para''(O)$, etc. This makes an extremely low probability of deriving the true parameters ($Para(O)$) because the decision space falls into an entire arithmetic domain.

Hence, we prove that continuously monitoring the inference process to collect more observations of morphed parameters does not increase the (low) attack probability of deriving the true parameters and scalars.

5.2 Inference Integrity Analysis

Attack assumption. Recall that we assume a third-party attacker that can hijack the client and manipulate any data outside enclave to perturb the inference process (§3.2). Specifically, we consider a *random perturbation attack* that randomly perturbs any observed data outside enclave. Next, we give ana-

lytical and experimental evaluation of the integrity guarantees of our *oblivious fingerprint* technique.

Hidden architecture for stealthiness. SOTER hides fingerprint challenges among ordinary queries to make detection stealthy. To achieve stealthiness, SOTER partitions a model by reserving *top-W* operators in enclave (Line 5 in Algorithm 2). By doing so, given a query, we hide the architecture information such that the adversary cannot determine the IRs (intermediate results transmitted from enclave to GPU) for every partitioned operator; If not doing so, assuming the first operator O_0 is partitioned to GPU, then for arbitrary query q, the adversary can get the IR of O_0 by calculating $O_0(q)$. Other IRs transmitted to O_0 are easily detected as fingerprint challenges, and the adversary would bypass our detection.

Fingerprint obliviousness evaluation. Even with stealthy fingerprint challenges, an adversary who monitors the entire inference process can still distinguish fixed challenges used for detection. Next, we evaluate the obliviousness of SOTER's dynamically derived fingerprints by analyzing the distribution of randomly sampled input to a given partitioned operator O.

Inspired from previous work [37], we analyzed the l_2 distance of two randomly sampled input (including both fingerprint challenges and IRs) sent from enclave to GPU, which should fits a normal (Gaussian) distribution [62, 77]. This is because, the IRs produced by a model are typically defined in a bounded space, i.e., elements of inputs are defined over a certain range of values [37]. Since we exclude black-box attacks here and assume queries are benign (§3.2), the input to O should be distributed in a natural and consistent manner.

Figure 7 depicts the l_2 distance distribution of SOTER's oblivious fingerprints and the straw-man fixed fingerprints. We observed that by using the associativity property to dynamically derive new fingerprints at runtime, the adversary can not distinguish the fingerprint challenges because the distance distribution between two random input is close to a normal distribution. Whereas, with fixed fingerprints, the adversary can observe a totally different distribution, which implies that fingerprint challenges exist and the adversary could stop attacks and evade our detection.

Detection probability. Next we prove our high probability to detect random perturbation attacks.

Denote a *perturbed* partitioned operator as O, the detection rate (probability of sending integrity challenges to O) as p, and the number of client input (queries) as N. Given that the adversary can only randomly guess whether an input to O is a challenge or not (due to the obliviousness property proved above), the probability of successfully detecting O's integrity breach after N queries is therefore $P(X = N) = 1 - (1 - t/2)^N$ (see Appendix A). With detection rate $p = 0.8$, SOTER can detect integrity breaches on partitioned operator with an overwhelming 99.39% probability of only $N = 10$ queries. This proof matches our evaluation in Figure 6.3.

6 Performance Evaluation

Testbed. We conducted all evaluation in our cluster on a GPU farm of 10 DELL R430 servers with 20 Nvidia RTX2080TI graphic cards. Each graphic card is connected to the host with PCIe 3.0. Each server is equipped with an Intel 2.60GHz E5-2690 V3 CPU, 64GB memory, and 40Gbps NIC. We ran SGX with the real Intel SGX hardware in our cluster.

Model and dataset. As shown in Table 2, we evaluated nine popular DNN models include a Multi-layer Perception (MLP) model, six Convolution Neural Network (CNN) models including AlexNet [43], DenseNet-121 [33], VGG-19 [76], ResNet-101 [30], NASNet [94] and PNASNet [50], a Recurrent Neural Network (RNN) WaveRNN [39] and a transformer model [2]. Among these models, the MLP model is purely composed by the same linear operators (*torch.nn.Linear*) and ReLU activation functions (*torch.nn.ReLU*); For the CNN models, AlexNet, DenseNet-121, VGG-19, and ResNet-101 are popular models that achieve high inference accuracy on computer vision tasks. In addition to these CNN models, we evaluated two large CNN models NASNet and PNASNet which are designed by recent *neural architecture search* (NAS) techniques [56]; WaveRNN is an efficient RNN model developed by Deepmind and Google for efficient speech-to-text transformations [39]; Transformer is the most popular model for natural language processing [2].

The parameters of the models were trained by using well-known datasets on PyTorch. We used ImageNet [14] for CV models (Image Classification tasks) and WMT16 EN-DE [1] for NLP models (Translation tasks), both of which are well-known large-scale training samples.

Baselines. We compared SOTER with MLcapsule [29], a popular SGX-based secure inference system at the client-side. MLcapsule shields the entire model in SGX and executes inference tasks only with CPU, providing a fully black-box guarantee to offloaded models but coming at a substantial cost on inference latency. Since MLcapsule is not open-sourced, we built a proof-of-concept implementation based on the descriptions in MLcapsule's paper [29]. We also compared SOTER with the state-of-the-art model partition approach eNNclave [69], a notable open-sourced inference system at the client-side that uses handcrafted model partition. Although DeepEnclave [28] uses SGX for model partition, we did not evaluate it because DeepEnclave has an orthogonal focus on protecting client's input data.

We divide our evaluation into three sections that answer the following questions:

§6.1 How efficient is SOTER compared with existing secure inference systems at the client-side?

§6.2 How accurate is SOTER compared with existing secure inference systems at the client-side?

§6.3 How robust is SOTER under integrity attacks?

Table 2: Overview of evaluated models and datasets.

Task	Pattern	Model	Param Size (MB)	Defaulte Input Tensor Shape	Dataset
Image Classification	MLP	MNIST-MLP (MLP) [45]	4.58	(1, 1, 28, 28)	MNIST [44]
	CNN	AlexNet (AN) [43]	233.08	(1, 3, 224, 224)	ImageNet [14]
		DenseNet-121 (DN) [33]	30.44		
		VGG-19 (VGG) [76]	548.05		
		ResNet-101 (RN) [30]	169.94		
		NASNet (NN) [94]	338.57	(1, 3, 331, 331)	
		PNASNet (PNN) [50]	328.68		
Translation	RNN	WaveRNN (WR) [39]	1254.64	512 tokens	WMT16 (En-De) [1]
	Transformer	Nvidia-Transformer (TF) [2]	1335.09	512 tokens	WMT16 (En-De) [1]

Table 3: Latency breakdown of SOTER ($\theta = 0.8$).

Models	Preprocessing (s)	Inference (ms)		
		SGX (CPU)	Kernel Switch	GPU
MLP	0.32	0.08	0.005	0.001
AlexNet	1.56	3.475	0.02	0.76
VGG-19	3.52	4.82	0.05	3.64
ResNet-101	2.29	2.30	0.06	7.10
DenseNet121	2.23	1.03	0.11	4.77
NASNet	8.23	274.43	11.21	27.71
PNASNet	9.47	293.61	12.38	29.41
WaveRNN	11.51	312.25	15.13	26.35
Transformer	12.41	334.97	15.60	34.35

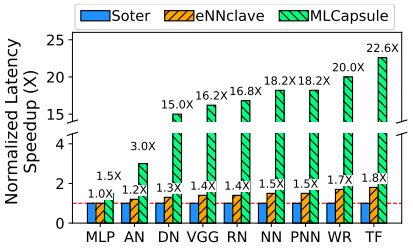


Figure 8: Normalized end-to-end inference latency comparison.

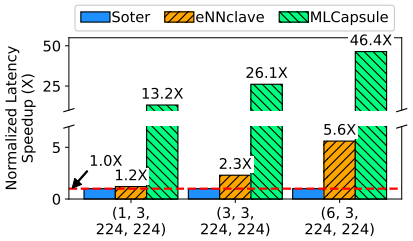


Figure 9: Inference latency with different input shape (batch_sise, channels, height, weight).

6.1 End-to-end Performance

End-to-end inference latency. We first investigated the end-to-end inference latency of SOTER and two baseline systems with nine models. All reported measurements are the average inference latency of 20 independent inferences on the same input. As shown in Figure 8, the partition-based secure inference systems including SOTER (execute inference tasks with both CPU and GPU) achieved better performance than MLcapsule (executes inference tasks only with CPU). This is because the CPU is at least one order of magnitude less efficient than GPU [80] in executing the inference tasks. Among all baseline systems, SOTER achieved the lowest inference latency for all models. Moreover, the latency gaps between SOTER and baseline systems grew larger as model / input size increases (shown in Figure 9). For instance, with a small DNN model (AlexNet), SOTER achieved $\sim 3.0\%$ lower inference latency on average than existing secure inference systems. With a larger DNN model (Transformer), SOTER achieved up to 22.6% lower inference latency on average than existing secure inference systems.

Latency breakdown. To understand SOTER's low inference latency, we broke down the time taken in different phases of an inference task in SOTER, and the results are reported in Table 3. It can be clearly observed that, for all models, with a 80% probability ($\theta = 0.8$) to partition an associative operator, SOTER securely offloaded most computation-expensive model layers to GPU and successfully downgraded inference latency compared to pure CPU inference, resulting in low end-to-end inference latency. Doing so requires SOTER to copy the model layer parameters from SGX to the GPU memory only in the preprocessing phase. During the inference phase, SOTER only copies intermediate results (much smaller than the model parameters) into the GPU memory, and copies the outputs back to the SGX enclave memory for each layer. Frequent memory copies between GPU and SGX may cause a substantial latency penalty. To amortize these memory copy overhead, SOTER batches several continuous operators together, and performs input/output memory copies only before and after computing the batch of operators.

Overall, SOTER can achieve a low inference latency, and the computation overhead (CPU computation in SGX and kernel switch) caused by enclaving the plaintext operators only increases sub-linearly with the model size.

6.2 Inference Accuracy

In this section, we evaluate the resulting inference accuracy of SOTER and existing secure inference systems. For each system, we compared the inference accuracy of the protected models (i.e., running some or all layers of the ML model in SGX enclaves to protect the model) and the corresponding white-box unprotected model (i.e., running the entire model on GPU without the protection of SGX).

SOTER and MLcapsule achieved zero accuracy drop, while

other partition-based systems caused inference accuracy drops from 1% to 5.5% for all evaluated CNN models. MLcapsule was able to achieve zero accuracy loss because it shields the entire model in the SGX enclave to protect the highest security guarantee. However, at the same time, the security guarantee of MLcapsule come at a steep price in performance, compared to untrusted alternatives (i.e., perform inference tasks on the GPU).

To achieve better performance, the partition-based secure inference systems outsource some inference computations to the untrusted GPU. However, existing partition-based inference system (i.e., eNNclave) takes a handcrafted approach to partition an offloaded model and tunes the parameters of the outsourced layers, leading to a drop in the inference accuracy. By safely morphing and partitioning computation-expensive operators to GPU for faster inference computing, and then recovers the real inference result in SGX, SOTER achieves stronger model confidentiality (§5.1) compared to eNNclave without sacrificing the inference accuracy.

6.3 Robustness to Integrity Breach Attacks

We evaluated the robustness of SOTER under the integrity breach attacks. We conducted an integrity attack by continuously and randomly tempering with the outputs of layers outside the enclave, and monitoring the number of fingerprints for SOTER to detect this attack. The evaluation results of 10K attacks are shown in Figure 10. All attacks were successfully detected, and 99.1% attacks were detected with less than 10 fingerprints while we incurred zero false positives. This result matches our theoretical analysis in §5.2.

7 Discussion

7.1 TEE-based handcrafted approaches

AI applications (e.g., face recognition [47], autopilot [8]) require both high performance (i.e., low inference latency and high accuracy) and strong privacy. Due to the lack of commonly-available trusted GPUs [5, 83], many existing systems propose protocols to empower secure model inference by co-designing an accelerator (e.g., GPU) with Trusted Execution Environments (e.g., Intel SGX [12]).

eNNclave [69] is a notable partition-based secure inference system. Unlike SOTER, eNNclave handcrafts a partition plan by manually identifying less sensitive model layers (operators), and then uses transfer learning techniques [71] to retrain these layers' parameters into function-similar, yet insensitive new parameters. Unfortunately, such handcrafted approach inevitably meets a generality bottleneck. Due to the heuristic nature (weak explainability) of complex models [21, 65, 89], manually determining the sensitivity for every operator (layer) could be extremely hard. If falsely selects a sensitive layer

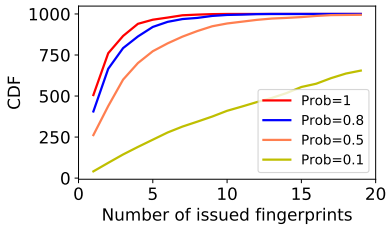


Figure 10: The CDF of the number of issued fingerprint challenges for catching 1000 random perturbation attacks with different issuing probability P.

and retrains it with transfer learning, the inference accuracy would severely downgrades.

DeepEnclave [28], like eNNclave, is another handcrafted model partition approach for secure inference. It uses knowledge distillation [11, 90] to hide manually identified sensitive layers. However, DeepEnclave is a cloud-side inference system with a totally different focus on protecting users' input queries, hence SOTER does not compare with DeepEnclave.

7.2 Limitations and Future Work

Model leakage from black-box attacks. Modern black-box attacks [6, 37, 58, 60] use inference APIs of an inferene service to learn private information of an offloaded model or even the training dataset, by means of the inference results of perturbed input queries. These attacks widely exist in either cloud-side or client-side model inference systems [29, 38, 53, 69], as inference systems inevitably open access to arbitrary queries.

Currently, there is no general defenses against black-box attacks other than query authentication and rate limiting [37, 58]. But fortunately, defenses against specifc attacks are emerging. For instance, Prada [37] uses query distance auditing to mitigate model extraction attacks with adversarial examples; Shokroi et al. [75] uses differential privacy to train DNNs that do not leak model owner's sensitive training dataset.

The guarantees of SOTER, like all prior secure inference systems targeting at protecting the model at the edge or the cloud, are complementary to those provided by any defenses. With sufficient engineering effort, these mitigations can be integrated into SOTER to provide even stronger privacy guarantees, and we leave it to future work.

Usage of Trusted Execution Environment. SOTER requires clients at the edge to be equipped with TEE (e.g., Intel SGX) due to the lack of commonly available trusted GPUs [5, 83]. SGX has been pervasively used in existing secure inference systems [29, 69] and is commonly available in modern Intel CPUs. While the inherited SGX vulnerabilities of sophisticated side-channel attacks based on timing or cache access patterns still exist [16, 84], we do not consider these attacks currently, and we believe these attacks can be mitigated by integrating with state-of-the-art defenses [16, 17, 67].

8 Conclusion

We present SOTER, the first secure inference protocol that ensures both strong confidentiality and integrity for general DNN models. SOTER partitions computation-expensive operators from TEE to the co-located untrusted GPU by morphing a fraction of associative operators' parameters and recovers the real execution results in TEE. Evaluation on notable client-side secure inference systems and all prevalent types of DNN models shows that SOTER is the first protocol that can achieve low inference latency while maintaining high accuracy at the same time. SOTER is general for placing proprietary DNN models on untrustworthy clients at the edge, and is especially suitable for security-critical and performance-sensitive AI applications such as autopilot and stock prediction. SOTER's source code is released on github.com/ndss22p16/soter.

Appendices

A SOTER's Detection probability

The contents...

References

[1] Wmt16. http://www.statmt.org/wmt16/translation-task.html, 2016.

[2] Transformer. https://ngc.nvidia.com/catalog/resources, 2020.

[3] Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.

[4] Anshul Aggarwal, Trevor E Carlson, Reza Shokri, and Shruti Tople. Soteria: In search of efficient neural networks for private inference. *arXiv preprint arXiv:2007.12934*, 2020.

[5] Aref Asvadishirehjini, Murat Kantarcioglu, and Bradley Malin. Goat: Gpu outsourcing of deep learning training with asynchronous probabilistic integrity verification inside trusted execution environment. *arXiv preprint arXiv:2010.08855*, 2020.

[6] Buse Gul Atli, Sebastian Szyller, Mika Juuti, Samuel Marchal, and N Asokan. Extraction of complex dnn models: Real threat or boogeyman? In *International Workshop on Engineering Dependable and Secure Machine Learning Systems*, pages 42–57. Springer, 2020.

[7] J Aumasson and L Merino. Sgx secure enclaves in practice–security and crypto review. *Black Hat*, 2016.

[8] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.

[9] Han Cai, Ligeng Zhu, and Song Han. Proxylessnas: Direct neural architecture search on target task and hardware. *arXiv preprint arXiv:1812.00332*, 2018.

[10] Tiago Carvalho, Edmar RS De Rezende, Matheus TP Alves, Fernanda KC Balieiro, and Ricardo B Sovat. Exposing computer generated images by eye's region classification via transfer learning of vgg19 cnn. In *2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 866–870. IEEE, 2017.

[11] Xu Cheng, Zhefan Rao, Yilan Chen, and Quanshi Zhang. Explaining knowledge distillation by quantifying the knowledge. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12925–12935, 2020.

[12] Victor Costan and Srinivas Devadas. Intel sgx explained. *IACR Cryptol. ePrint Arch.*, 2016(86):1–118, 2016.

[13] Piali Das, Nikita Ivkin, Tanya Bansal, Laurence Rouesnel, Philip Gautier, Zohar Karnin, Leo Dirac, Lakshmi Ramakrishnan, Andre Perunicic, Iaroslav Shcherbatyi, et al. Amazon sagemaker autopilot: a white box automl solution at scale. In *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, pages 1–7, 2020.

[14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.

[15] Shuiguang Deng, Hailiang Zhao, Weijia Fang, Jianwei Yin, Schahram Dustdar, and Albert Y Zomaya. Edge intelligence: The confluence of edge computing and artificial intelligence. *IEEE Internet of Things Journal*, 7(8):7457–7469, 2020.

[16] Ghada Dessouky, Tommaso Frassetto, and Ahmad-Reza Sadeghi. Hybcache: Hybrid side-channel-resilient caches for trusted execution environments. In *29th {USENIX} Security Symposium ({USENIX} Security 20)*, pages 451–468, 2020.

[17] Anuj Dubey, Rosario Cammarota, Vikram Suresh, and Aydin Aysu. Guarding machine learning hardware against physical side-channel attacks. *arXiv preprint arXiv:2109.00187*, 2021.

[18] S Durga, Esther Daniel, and S Deepakanmani. Dnn-based decision support system for ecg abnormalities. In *2nd EAI International Conference on Big Data Innovation for Sustainable Cognitive Computing*, pages 331–338. Springer, 2021.

[19] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *The Journal of Machine Learning Research*, 20(1):1997–2017, 2019.

[20] Simon Fong, Suash Deb, and Xin-she Yang. How meta-heuristic algorithms contribute to deep learning in the hype of big data analytics. In *Progress in intelligent computing techniques: theory, practice, and applications*, pages 3–25. Springer, 2018.

[21] Krishna Gade, Sahin Cem Geyik, Krishnaram Kenthapadi, Varun Mithal, and Ankur Taly. Explainable ai in industry. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3203–3204, 2019.

[22] Yongsheng Gao and Maylor KH Leung. Face recognition using line edge map. *IEEE transactions on pattern analysis and machine intelligence*, 24(6):764–779, 2002.

[23] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[24] Google. Cloud speech-to-text api -language support. https://cloud.google.com/speech-to-text/docs/languages.

[25] Google. Google lens. https://lens.google/.

[26] T Gopalakrishnan, D Ruby, Fadi Al-Turjman, Deepak Gupta, Irina V Pustokhina, Denis A Pustokhin, and K Shankar. Deep learning enabled data offloading with cyber attack detection model in mobile edge computing systems. *IEEE Access*, 8:185938–185949, 2020.

[27] Luca Greco, Pierluigi Ritrovato, and Fatos Xhafa. An edge-stream computing infrastructure for real-time analysis of wearable sensors data. *Future Generation Computer Systems*, 93:515–528, 2019.

[28] Zhongshu Gu, Heqing Huang, Jialong Zhang, Dong Su, Ankita Lamba, Dimitrios Pendarakis, and Ian Molloy. Securing input data of deep learning inference systems via partitioned enclave execution. *arXiv preprint arXiv:1807.00969*, 2018.

[29] Lucjan Hanzlik, Yang Zhang, Kathrin Grosse, Ahmed Salem, Maximilian Augustin, Michael Backes, and Mario Fritz. Mlcapsule: Guarded offline deployment of machine learning as a service. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3300–3309, 2021.

[30] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[31] David A Hensher and William H Greene. The mixed logit model: the state of practice. *Transportation*, 30(2):133–176, 2003.

[32] Chuang Hu, Wei Bao, Dan Wang, and Fengming Liu. Dynamic adaptive dnn surgery for inference acceleration on the edge. In *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, pages 1423–1431. IEEE, 2019.

[33] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.

[34] Insu Jang, Adrian Tang, Taehoon Kim, Simha Sethumadhavan, and Jaehyuk Huh. Heterogeneous isolated execution for commodity gpus. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 455–468, 2019.

[35] Joanna Jaworek-Korjakowska, Pawel Kleczek, and Marek Gorgon. Melanoma thickness prediction based on convolutional neural network with vgg-19 model transfer learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 0–0, 2019.

[36] Jianyu Jiang, Xusheng Chen, TszOn Li, Cheng Wang, Tianxiang Shen, Shixiong Zhao, Heming Cui, Cho-Li Wang, and Fengwei Zhang. Uranus: Simple, efficient sgx programming and its applications. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, pages 826–840, 2020.

[37] Mika Juuti, Sebastian Szyller, Samuel Marchal, and N Asokan. Prada: protecting against dnn model stealing attacks. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 512–527. IEEE, 2019.

[38] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. {GAZELLE}: A low latency framework for secure neural network inference. In *27th {USENIX} Security Symposium ({USENIX} Security 18)*, pages 1651–1669, 2018.

[39] Nal Kalchbrenner, Erich Elsen, Karen Simonyan, Seb Noury, Norman Casagrande, Edward Lockhart, Florian Stimberg, Aaron Oord, Sander Dieleman, and Koray

Kavukcuoglu. Efficient neural audio synthesis. In *International Conference on Machine Learning*, pages 2410–2419. PMLR, 2018.

[40] David Kaplan, Jeremy Powell, and Tom Woller. Amd memory encryption. *White paper*, 2016.

[41] Rafał Kozik, Michał Choraś, Massimo Ficco, and Francesco Palmieri. A scalable distributed machine learning approach for attack detection in edge computing environments. *Journal of Parallel and Distributed Computing*, 119:18–26, 2018.

[42] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[43] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25:1097–1105, 2012.

[44] Yann LeCun. The mnist database of handwritten digits. *http://yann. lecun. com/exdb/mnist/*, 1998.

[45] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[46] En Li, Zhi Zhou, and Xu Chen. Edge intelligence: On-demand deep learning model co-inference with device-edge synergy. In *Proceedings of the 2018 Workshop on Mobile Edge Communications*, pages 31–36, 2018.

[47] Jinyu Li, Dong Yu, Jui-Ting Huang, and Yifan Gong. Improving wideband speech recognition using mixed-bandwidth training data in cd-dnn-hmm. In *2012 IEEE Spoken Language Technology Workshop (SLT)*, pages 131–136. IEEE, 2012.

[48] Qianmu Li, Shunmei Meng, Sainan Zhang, Jun Hou, and Lianyong Qi. Complex attack linkage decision-making in edge computing networks. *IEEE access*, 7:12058–12072, 2019.

[49] Yandong Li, Lijun Li, Liqiang Wang, Tong Zhang, and Boqing Gong. Nattack: Learning the distributions of adversarial examples for an improved black-box attack on deep neural networks. In *International Conference on Machine Learning*, pages 3866–3876. PMLR, 2019.

[50] Chenxi Liu, Barret Zoph, Maxim Neumann, Jonathon Shlens, Wei Hua, Li-Jia Li, Li Fei-Fei, Alan Yuille, Jonathan Huang, and Kevin Murphy. Progressive neural architecture search. In *Proceedings of the European conference on computer vision (ECCV)*, pages 19–34, 2018.

[51] Xiaolong Ma, Fu-Ming Guo, Wei Niu, Xue Lin, Jian Tang, Kaisheng Ma, Bin Ren, and Yanzhi Wang. Pconv: The missing but desirable sparsity in dnn weight pruning for real-time execution on mobile devices. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 5117–5124, 2020.

[52] Lubin Meng, Chin-Teng Lin, Tzyy-Ping Jung, and Dongrui Wu. White-box target attack for eeg-based bci regression problems. In *International conference on neural information processing*, pages 476–488. Springer, 2019.

[53] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. Delphi: A cryptographic inference service for neural networks. In *29th {USENIX} Security Symposium ({USENIX} Security 20)*, pages 2505–2522, 2020.

[54] Mohammad H Mofrad, Adam Lee, and Spencer L Gray. Leveraging intel sgx to create a nondisclosure cryptographic library. *arXiv preprint arXiv:1705.04706*, 2017.

[55] Robert C Moore and Will Lewis. Intelligent selection of language model training data. 2010.

[56] Stefan Nastic, Thomas Rausch, Ognjen Scekic, Schahram Dustdar, Marjan Gusev, Bojana Koteska, Magdalena Kostoska, Boro Jakimovski, Sasko Ristov, and Radu Prodan. A serverless real-time data analytics platform for edge computing. *IEEE Internet Computing*, 21(4):64–71, 2017.

[57] Lucien KL Ng, Sherman SM Chow, Anna PY Woo, Donald PH Wong, and Yongjun Zhao. Goten: Gpu-outsourcing trusted execution of neural network training. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 14876–14883, 2021.

[58] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Prediction poisoning: Towards defenses against dnn model stealing attacks. *arXiv preprint arXiv:1906.10908*, 2019.

[59] Daniele Palossi, Antonio Loquercio, Francesco Conti, Eric Flamand, Davide Scaramuzza, and Luca Benini. A 64-mw dnn-based visual navigation engine for autonomous nano-drones. *IEEE Internet of Things Journal*, 6(5):8357–8371, 2019.

[60] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, pages 506–519, 2017.

[61] Li Pengcheng, Jinfeng Yi, and Lijun Zhang. Query-efficient black-box attack by active learning. In *2018 IEEE International Conference on Data Mining (ICDM)*, pages 1200–1205. IEEE, 2018.

[62] Johan Philip. *The probability distribution of the distance between two random points in a box*. KTH mathematics, Royal Institute of Technology, 2007.

[63] Sandro Pinto and Nuno Santos. Demystifying arm trustzone: A comprehensive survey. *ACM Computing Surveys (CSUR)*, 51(6):1–36, 2019.

[64] Matt Post. A call for clarity in reporting bleu scores. *arXiv preprint arXiv:1804.08771*, 2018.

[65] Alun Preece. Asking 'why'in ai: Explainability of intelligent systems–perspectives and challenges. *Intelligent Systems in Accounting, Finance and Management*, 25(2):63–72, 2018.

[66] Hussam Qassim, Abhishek Verma, and David Feinzimer. Compressed residual-vgg16 cnn model for big data places image recognition. In *2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 169–175. IEEE, 2018.

[67] Sajin Sasy, Sergey Gorbunov, and Christopher W Fletcher. Zerotrace: Oblivious memory primitives from intel sgx. In *NDSS*, 2018.

[68] Takami Sato, Junjie Shen, Ningfei Wang, Yunhan Jack Jia, Xue Lin, and Qi Alfred Chen. Security of deep learning based lane keeping system under physical-world adversarial attack. *arXiv preprint arXiv:2003.01782*, 2020.

[69] Alexander Schlögl and Rainer Böhme. ennclave: Offline inference with model confidentiality. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, pages 93–104, 2020.

[70] Rico Sennrich, Barry Haddow, and Alexandra Birch. Edinburgh neural machine translation systems for wmt 16. *arXiv preprint arXiv:1606.02891*, 2016.

[71] Mehmet Saygin Seyfioglu, Baris Erol, Sevgi Zubeyde Gurbuz, and Moeness G Amin. Dnn transfer learning from diversified micro-doppler for motion classification. *IEEE Transactions on Aerospace and Electronic Systems*, 55(5):2164–2180, 2018.

[72] Jiawei Shao and Jun Zhang. Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems. In *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*, pages 1–6. IEEE, 2020.

[73] Tianxiang Shen, Jianyu Jiang, Yunpeng Jiang, Xusheng Chen, Ji Qi, Shixiong Zhao, Fengwei Zhang, Xiapu Luo, and Heming Cui. Daenet: Making strong anonymity scale in a fully decentralized network. *IEEE Transactions on Dependable and Secure Computing*, 2021.

[74] Kilho Shin and Tetsuji Kuboyama. A generalization of haussler's convolution kernel: mapping kernel. In *Proceedings of the 25th international conference on Machine learning*, pages 944–951, 2008.

[75] Reza Shokri and Vitaly Shmatikov. Privacy-preserving deep learning. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, pages 1310–1321, 2015.

[76] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[77] Luis Antonio Santaló Sors and Luis A Santaló. *Integral geometry and geometric probability*. Cambridge university press, 2004.

[78] Srikanth Tammina. Transfer learning using vgg-16 with deep convolutional neural network for classifying images. *International Journal of Scientific and Research Publications (IJSRP)*, 9(10):143–150, 2019.

[79] Sasha Targ, Diogo Almeida, and Kevin Lyman. Resnet in resnet: Generalizing residual architectures. *arXiv preprint arXiv:1603.08029*, 2016.

[80] Florian Tramer and Dan Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. *arXiv preprint arXiv:1806.03287*, 2018.

[81] Tuyen X Tran, Mohammad-Parsa Hosseini, and Dario Pompili. Mobile edge computing: Recent efforts and five key research directions. *IEEE COMSOC MMTC Commun.-Frontiers*, 2017.

[82] Chia-Che Tsai, Donald E Porter, and Mona Vij. Graphene-sgx: A practical library {OS} for unmodified applications on {SGX}. In *2017 {USENIX} Annual Technical Conference ({USENIX}{ATC} 17)*, pages 645–658, 2017.

[83] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. Graviton: Trusted execution environments on gpus. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, pages 681–696, 2018.

[84] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A Gunter. Leaky cauldron on the dark land: Understanding memory side-channel hazards in

sgx. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2421–2434, 2017.

[85] Wenxuan Wang, Bangjie Yin, Taiping Yao, Li Zhang, Yanwei Fu, Shouhong Ding, Jilin Li, Feiyue Huang, and Xiangyang Xue. Delving into data: Effectively substitute training for black-box attack. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4761–4770, 2021.

[86] Yajie Wang, Yu-an Tan, Wenjiao Zhang, Yuhang Zhao, and Xiaohui Kuang. An adversarial attack on dnn-based black-box object detectors. *Journal of Network and Computer Applications*, 161:102634, 2020.

[87] Jongho Won, Seung-Hyun Seo, and Elisa Bertino. A secure shuffling mechanism for white-box attack-resistant unmanned vehicles. *IEEE Transactions on Mobile Computing*, 19(5):1023–1039, 2019.

[88] Yong Wu, Xiao Qin, Yonghua Pan, and Changan Yuan. Convolution neural network based transfer learning for classification of flowers. In *2018 IEEE 3rd international conference on signal and image processing (ICSIP)*, pages 562–566. IEEE, 2018.

[89] Roman V Yampolskiy. Unexplainability and incomprehensibility of ai. *Journal of Artificial Intelligence and Consciousness*, 7(02):277–291, 2020.

[90] Junho Yim, Donggyu Joo, Jihoon Bae, and Junmo Kim. A gift from knowledge distillation: Fast optimization, network minimization and transfer learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4133–4141, 2017.

[91] Chao Zhang and Philip C Woodland. Parameterised sigmoid and relu hidden activation functions for dnn acoustic modelling. In *Sixteenth Annual Conference of the International Speech Communication Association*, 2015.

[92] Jiliang Zhang, Wuqiao Chen, and Yuqi Niu. Deepcheck: A non-intrusive control-flow integrity checking based on deep learning. *arXiv preprint arXiv:1905.01858*, 2019.

[93] Qiao Zhang, Chunsheng Xin, and Hongyi Wu. Gala: Greedy computation for linear algebra in privacy-preserved neural networks. *arXiv preprint arXiv:2105.01827*, 2021.

[94] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V Le. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8697–8710, 2018.

[95] Pengfei Zuo, Yu Hua, Ling Liang, Xinfeng Xie, Xing Hu, and Yuan Xie. Sealing neural network models in secure deep learning accelerators. *arXiv preprint arXiv:2008.03752*, 2020.